



Agentic Enterprise Blueprint



Agentic Taskforce

Agentic Platform and Models

Agentic Defense

Agentic Data Cloud

AI Hypercomputer



Agentic Enterprise Blueprint

El Sistema Operativo de una empresa agéntica

Los 4 pilares de Agent Platform: Build · Scale · Govern · Optimize

BUILD

SCALE

GOVERN

OPTIMIZE

Hacia un mundo con miles de agentes por empresa

>1.600

agentes por empresa
para fines de 2026

IBM Think 2026

75%

de clientes de Google
Cloud
usan productos de IA

Google Cloud Next '26

40%

de apps empresariales
con agentes integrados
para fines de 2026

Gartner, 2026

70%

de ejecutivos admiten que
su gobernanza de IA no
alcanza para esta escala

IBM Enterprise Survey, 2026

GE Appliances

800+

agentes

KPMG

100+

agentes en 1 mes

Virgin Voyages

1.000+

agentes especializados

Vodafone

€100M

en ahorros para 2030

"Today, that future is in-production — the Agentic Enterprise is real."

— Thomas Kurian, CEO Google Cloud · Google Cloud Next '26

El agente funciona en el notebook. ¿Y en producción?

Lo que funciona en el notebook

- Gemini responde correctamente a las queries
- Las tools ejecutan SQL sin errores
- El flujo ReAct produce resultados válidos
- Los tests locales pasan

Lo que falta para producción

- ¿Quién es el agente?** Sin identidad corporativa ni audit trail
- ¿Dónde está registrado?** Sin control de versiones ni rollback
- ¿Cómo se conecta?** Sin gateway — superficie de ataque expuesta
- ¿Qué puede responder?** Sin filtros de contenido ni políticas RAI
- ¿Cómo sabemos si falla?** Sin observabilidad ni evals automatizados

Estos 5 problemas son los que resuelven los 4 pilares de Agent Platform.

◆ Gemini Enterprise Agent Platform

Build

New

Agent Development Kit

3P agent frameworks

New

Agent Studio

Agent Garden

Gemini API and Model Garden

Gemini models

3P and open models

Model training

Model inference

Tools, data, and other agents

A2A

Grounding

RAG

MCP

RAG

API and connectors

A2UI

AP2 and UCP

Cloud Marketplace

Scale

GA

Agent Runtime

GA

Agent Sessions

GA

Agent Sandbox

GA

Agent Memory Bank

Govern

New

Agent Gateway

GA

Agent Identity

New

Agent Registry

New

Agent Anomaly Detection

Model Armor

Agent Policy

New

Agent Security

Agent Compliance

Optimize

New

Agent Evaluation

New

Agent Simulation

New

Agent Observability

New

Agent Optimizer

Los 4 pilares de Agent Platform

BUILD

- **ADK** — framework Python code-first
- **A2A** — protocolo inter-agentes
- **Agents CLI + DAKit** — scaffolding y workspace de datos
- **Agent Studio** — lienzo visual low-code **EA**

SCALE

- **Agent Runtime** — serverless, cold start sub-segundo
- **Memory Bank + Sessions** — persistencia entre conversaciones
- **Agent Sandbox** — ejecución segura de código
- **Flujos multi-día** — autonomía extendida + checkpoint

GOVERN

- **Agent Identity** — SA dedicada por agente
- **Model Armor** — filtros input/output inline
- **Agent Gateway** — red privada sin exposición **EA**
- **Agent Registry** — inventario centralizado de la flota **EA**

OPTIMIZE

- **Agent Evaluation** — evals + LLM-as-judge **EA**
- **Agent Observability** — traces, logs y métricas **EA**
- **Agent Simulation** — stress tests con usuarios sintéticos **EA**

ADK — Agent Development Kit

Qué provee ADK

- **Framework Python** — tools declarativas, tipado, Gemini Enterprise nativa
- **Loop ReAct built-in** — razona → actúa → observa → repite
- **Multi-agent nativo** — orquestador delega subagentes vía A2A
- **Callbacks de control** — before/after_model_callback para interceptar
- **Modelos intercambiables** — Gemini, Claude, Llama — mismo código
- **SDKs Python y Java** — flujos síncronos y asíncronos

Tool declarativa en ADK

```
@tool
def execute_sql(query: str) -> str:
    """Ejecuta SQL en BigQuery.
    Usa nombres exactos del catálogo."""

    client = bigquery.Client()

    return client.query(query).to_dataframe()
```

ADK separa la lógica del agente del código de negocio. Es el estándar de Google para agentes production-ready.

Protocolo A2A y arquitecturas multi-agent

Agent-to-Agent (A2A)

Protocolo estándar de Google para que agentes se comuniquen entre sí — independientemente del framework o lenguaje con que fueron construidos.

- Orquestador delega tareas a subagentes especializados
- Cada agente tiene su propia identidad y scope de tools
- Define cómo se pasan contexto, resultados y errores
- Compatible con ADK, LangGraph y frameworks externos

Patrón multi-agent en ADK

Agente Orquestador (Supervisor)



Agent Garden provee templates multi-agent listos: facturas, análisis financiero, soporte IT.

Agents CLI + Data Agent Kit

Agents CLI

Herramienta de línea de comandos para scaffolding, testing y deploy de agentes ADK.

```
$ agents new mi-agente  
  
$ agents run --local  
  
$ agents deploy --target=cloud-run  
  
$ agents eval --dataset=test_cases.jsonl
```

Data Agent Kit

Suite portable de skills, MCP tools y plugins para el IDE del developer — VS Code, Gemini CLI, Claude Code.

Workspace nativo de datos sin salir del editor. No es el SDK del agente — ese es ADK.

→ **IDEs soportados** — VS Code · Gemini CLI · Claude Code

→ **BigQuery + KC** — consulta SQL y navega esquemas sin salir del editor

→ **RAG Engine** — búsqueda semántica y grounding desde el workspace

→ **No es ADK** — ADK construye el agente — DAKit es para el developer

Agent Studio — lienzo visual de bajo código

La segunda ruta de BUILD

ADK es la ruta code-first para ingeniería. Agent Studio es un lienzo visual donde equipos de negocio y analytics diseñan agentes sin escribir código Python.

- El usuario describe en lenguaje natural qué quiere que haga el agente
- Studio genera la estructura con las tools apropiadas
- No es un generador de código — es un diseñador visual de flujos agénticos
- Respaldo por ADK bajo el capó

Clave técnica

Los agentes creados en Studio son compatibles con el mismo Agent Runtime, Agent Registry y capa GOVERN que los agentes code-first.

No hay dos plataformas — hay dos interfaces para la misma plataforma.

Beneficio organizacional

- Los casos simples los construye el equipo de negocio
- Los complejos, ingeniería en ADK
- Ambos con la misma infraestructura y gobernanza
- Agent Studio mueve el cuello de botella de pedidos a ingeniería

Agent Runtime — el ambiente gestionado de Agent Platform

Qué es Agent Runtime

Ambiente serverless totalmente gestionado para desplegar y escalar agentes en producción — sin gestión de infraestructura subyacente.

- **Cold starts en subsegundos** — provisioning de nuevos agentes en segundos
- **Integración nativa 1er nivel** — Gemini Enterprise, Agent Registry, Memory, Sessions
- **Soporte multi-framework** — ADK, LangGraph, LangChain, LlamaIndex, AG2, custom
- **Flujos de larga duración** — agentes autónomos corriendo días con estado persistente
- **Agent Sandbox integrado** — ejecución segura de código generado por el modelo

Capacidades de seguridad y gobernanza

- **Agent Identity nativa** — identidad SPIFFE + X.509 por agente, vinculada al ciclo de vida
- **Agent Gateway integrado** — routing seguro y gobernado, sin IP pública expuesta
- **Observabilidad agent-aware** — traces de razonamiento ReAct, métricas específicas de agentes
- **Memory Bank y Sessions** — bundled — sin configuración adicional
- **Private Service Connect** — interfaces VPC para conectividad completamente privada

Agent Runtime es el camino recomendado para producción en Agent Platform. Cloud Run y GKE son infraestructura genérica — no incluyen las capacidades nativas de la plataforma.

Agent Memory y Sessions

Memory Bank

- **Memoria a largo plazo** — el agente recuerda preferencias y contexto entre sesiones
- **Perfiles de Memoria** — cada usuario tiene un perfil personalizado gestionado
- **Sin configuración extra** — integrado en Agent Engine — sin base de datos adicional
- **Caso NowAgent Analytics** — Carlos Mendoza (CFO NovaMart) — el agente carga su perfil al inicio sin repetir el briefing

Agent Sessions

- **Historial vinculado** — vinculado a registros del sistema (CRM, tickets, ITSM) vía IDs personalizados
- **Streaming bidireccional** — protocolo WebSocket para interacciones en tiempo real con interrupción del usuario
- **Human-in-the-loop** — approval gates nativos antes de acciones irreversibles
- **Multi-turn context** — el agente mantiene contexto completo de la conversación sin repetición de datos

"El CFO pregunta en el Día 1. Al Día 2, sesión nueva, el agente ya sabe quién es y cuál es el contexto — sin repetir el briefing." — NowAgent Analytics, NovaMart

Agentes de larga duración y ejecución segura

Agent Sandbox

- Entorno hardened para ejecutar código generado por el modelo
- Aislamiento de red y sistema de archivos — el código del agente no puede comprometer el entorno host
- Para agentes de data science que generan y ejecutan Python dinámicamente

Flujos multi-día

- Agentes que corren de forma autónoma durante horas o días sin supervisión continua
- Checkpoint automático — el agente puede suspenderse y resumirse sin perder contexto
- Casos: auditorías masivas, migraciones, análisis de corpus documental completo

Bidirectional Streaming

- Protocolo WebSocket — el usuario puede interrumpir al agente en tiempo real
- El agente puede solicitar confirmación humana en puntos críticos del flujo
- Human-in-the-loop nativo: approval gates antes de acciones irreversibles

El problema de gobernanza en escala

>1.600

agentes por empresa
para fines de 2026

IBM Think 2026

70%

de ejecutivos admiten que
su gobernanza de IA
no alcanza para esta
escala

IBM Enterprise Survey, 2026

40%

de apps empresariales
con agentes integrados
para fines de 2026

Gartner, 2026

16B

tokens/minuto procesados
en la plataforma
de Google Cloud

Google Cloud Next '26

SHADOW AI

Agentes sin registro, sin
identidad, sin audit trail

SIN CONTROL

Sin gateway — cada
agente abre su propia
superficie de ataque

CERO TRAZABILIDAD

Imposible saber qué
agente tomó qué decisión

FUGA DE DATOS

Sin filtros de output — el
agente puede exfiltrar
información sensible

Los 4 componentes de Govern

Agent Identity

Identidad propia por agente

Cada agente opera con una identidad propia — SPIFFE+X.509 (nativo de la plataforma) o SA dedicada por agente. En NowAgent Analytics usamos SA dedicada: trazabilidad completa en Cloud Audit Logs, IAM específico por agente, revocable sin apagar el runtime.

Agent Registry

Inventario centralizado

Fuente única de verdad para todos los agentes de la organización. Evita herramientas huérfanas y versiones descontroladas. Si no está en el Registry, no debería poder invocar tools enterprise.

Agent Gateway

Red privada sin exposición

El agente no tiene IP pública ni endpoint expuesto. Control centralizado del tráfico con políticas de seguridad consistentes. Pipeline: AuthzPolicy → AuthzExtension → Model Armor → forward o 403.

Model Armor

Filtros inline de contenido

Inspección de input (prompt injection, jailbreak, URIs maliciosas) y output (data poisoning, fuga de datos, violaciones RAI). Si activa: log en Cloud Logging + respuesta de política.

Agent Identity — quién es el agente en producción

Lo que ofrece la plataforma

- **SPIFFE ID nativo** — identidad criptográfica por agente, vinculada a su ciclo de vida
- **Certificado X.509 auto-gestionado** — provisionado y renovado por Google cada 24 horas sin intervención
- **Tokens bound al certificado** — no reutilizables fuera del runtime — sin riesgo de replay
- **DPoP por defecto** — Demonstrable Proof of Possession — previene token theft

Nuestra implementación — NowAgent Analytics

Service Account dedicada por agente — trazabilidad completa en Cloud Audit Logs sin compartir identidad con otros workloads.

Identity:

```
nowagent-identity@coe-ds-sandbox-  
mzabaljauregui.iam.gserviceaccount.com
```

Resource:

```
//aiplatform.googleapis.com/projects/638115672237  
/locations/us-central1/reasoningEngines/3146057359567093760
```

- ✓ Cada llamada a BigQuery, KC y RAG Engine queda atribuida a esta SA
- ✓ IAM específico por agente — revocable sin apagar el agente
- ✓ Lo que IT Security necesita para dar el visto bueno en producción

Agent Registry — fuente de verdad de la flota

Deploy immutable

- Cada deploy genera un resource ID único con timestamp, versión de código y configuración de herramientas
- Nunca se sobrescribe — el historial completo siempre está disponible

```
reasoningEngines/  
3146057359567093760
```

Auditoría por versión

- Saber exactamente qué versión respondió una query en una fecha dada
- Reproducible y atribuible — compliance y debugging en producción
- Integrado con Cloud Audit Logs e IAM

Rollback controlado

- La versión anterior sigue viva en Agent Runtime — nunca fue destruida al hacer el nuevo deploy
- Rollback = redirigir el tráfico al resource ID anterior en el registry, sin tocar código
- Catálogo unificado de agentes, tools y MCP servers

Agent Registry + protocolo A2A/MCP = un catálogo único de agentes, herramientas y MCP servers — con control de versiones para toda la organización.

Agent Gateway — red privada sin exposición

¿Qué resuelve?

- **Sin IP pública** — el agente no tiene endpoint expuesto a internet
- **Control centralizado** — un único punto de entrada para todo el tráfico hacia agentes
- **Políticas consistentes** — las reglas se aplican uniformemente a todos los agentes
- **Rate limiting** — protección contra abuso y consumo descontrolado de cuota
- **VPC nativa** — el tráfico nunca sale de la red privada de la empresa

Flujo de una request entrante

Request entrante

- ↓ **AuthzPolicy** (agw-aisecurity)
- ↓ **AuthzExtension** (evaluación de contexto)
- ↓ **Model Armor API** (filtro de contenido)
 - MATCH** → **403 Forbidden**
 - NO_MATCH** → **Forward al agente**
- ↓ **Agente** (red privada, sin IP pública)

Model Armor — filtros inline de input y output

INPUT — before_model_callback

- ✗ Prompt injection — instrucciones maliciosas en el input
- ✗ Jailbreak attempts — sobrecribir el system prompt
- ✗ URIs maliciosas — links a contenido dañino o exfiltración
- ✗ Social engineering — patrones de manipulación del modelo
- ✗ PII no autorizado — datos sensibles que no deben procesarse

OUTPUT — after_model_callback

- ✗ Jailbreak confirmado — el modelo fue manipulado
- ✗ Data poisoning — output para corromper decisiones
- ✗ Fuga de información sensible — PII o datos propietarios
- ✗ Violaciones de políticas RAI — contenido no alineado
- ✗ Exfiltración encubierta — datos en respuestas normales

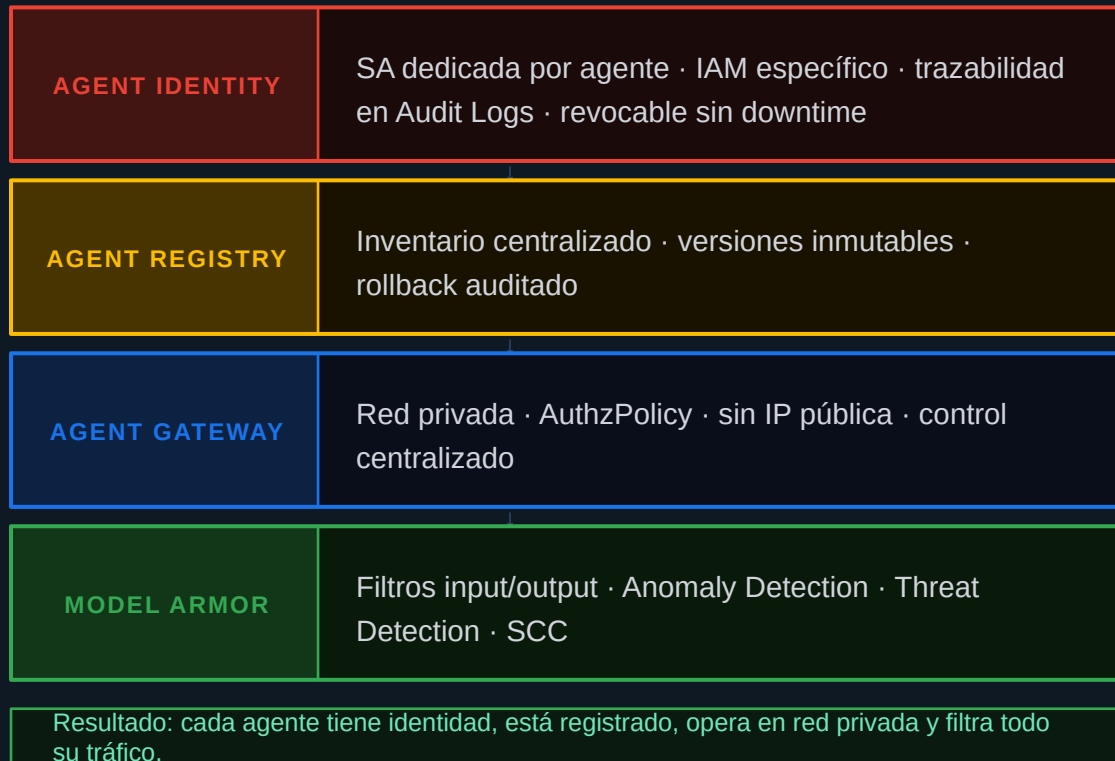
Si un filtro activa: el evento queda en Cloud Logging · el agente responde con mensaje de política · la respuesta nunca llega al usuario sin pasar los filtros.

Próximamente: Semantic Governance Policies — políticas en lenguaje natural desde la consola, sin callbacks de código.

Sin Model Armor vs Con Model Armor — 7 vectores de ataque

Vector de ataque	Sin Model Armor	Con Model Armor
Prompt injection del usuario	✗ Expuesto — ataque llega al modelo sin filtro	✓ Bloqueado — log + respuesta de política
Jailbreak del system prompt	✗ Expuesto — ataque llega al modelo sin filtro	✓ Bloqueado — log + respuesta de política
URLs maliciosas en input	✗ Expuesto — ataque llega al modelo sin filtro	✓ Bloqueado — log + respuesta de política
Exfiltración de datos en output	✗ Expuesto — ataque llega al modelo sin filtro	✓ Bloqueado — log + respuesta de política
Data poisoning de respuestas	✗ Expuesto — ataque llega al modelo sin filtro	✓ Bloqueado — log + respuesta de política
Violación de políticas RAI	✗ Expuesto — ataque llega al modelo sin filtro	✓ Bloqueado — log + respuesta de política
PII no autorizado procesado	✗ Expuesto — ataque llega al modelo sin filtro	✓ Bloqueado — log + respuesta de política

Stack de confianza completo



"Agent Platform enables this innovation to scale responsibly by combining deterministic business rules with probabilistic reasoning — making AI a trusted operational capability, not just a productivity tool."

Matt Olson
Chief Innovation Officer
Burns & McDonnell

Framework de evaluación de agentes

Test Cases y pipeline de evals (ADK)

- **Test cases en JSONL** — input + expected output + expected tools — definidos antes del deploy
- **agents eval** — ejecuta el agente contra todos los casos y produce reporte de métricas
- **Tool call accuracy** — ¿invocó las herramientas correctas en el orden correcto?
- **Response quality** — LLM-as-judge evalúa la calidad según rúbrica definida
- **Regression testing** — gate obligatorio antes de cada deploy — si baja una métrica, bloquea

Tipos de métricas

Determinísticas (automáticas)

ROUGE, BLEU, exact match — baja latencia, alta objetividad, para comparación contra ground truth.

LLM-as-Judge

Un modelo de alta capacidad evalúa helpfulness, tono, razonamiento y adherencia a instrucciones. Escala sin respuesta única correcta.

Trajectory Evaluation

Verifica la secuencia completa de pasos del agente — no solo el output final, sino el camino de razonamiento.

Un agente que pasa evals locales pero falla en producción indica un problema de distribución de datos — no de código.

Agent Observability — trazabilidad en producción

Qué se observa en Agent Engine

Traces

cada paso del flujo ReAct: tool invocada, input, output, latencia, tokens

Logs

Cloud Logging captura eventos de Model Armor, errores de tools y respuestas

Métricas

latencia p50/p95/p99, success rate, token usage, tool calls por sesión

Sessions

sesiones activas, turns promedio, usuarios únicos, distribución de temas

Feedback loop en producción

1

Monitor — Alertas si latencia o error rate sube sobre baseline

2

Diagnose — ¿Falla en tool? ¿Contexto insuficiente? ¿Prompt drift?

3

Fix + Eval — Corregir, correr evals, validar que las métricas no bajaron

4

Deploy nuevo — Deploy al Agent Registry — nuevo resource ID, nueva versión

5

Rollback si falla — Registry mantiene versión anterior, rollback en segundos

Agent Simulation — stress tests con usuarios sintéticos

Usuarios sintéticos

Perfiles realistas

CFO consultando revenue, auditor buscando inconsistencias, usuario malicioso con prompt injection

Escala masiva

cientos de interacciones simultáneas — sin equipo de QA humano

Previo al deploy

si el agente falla contra usuarios sintéticos, el deploy no avanza

Integrado en CI/CD

como load testing para microservicios, pero para comportamiento probabilístico

Tipos de escenarios

Happy path

flujos normales bien especificados — línea base de calidad antes de cubrir edge cases.

Edge cases

preguntas ambiguas, contexto contradictorio, tools que devuelven error, latencia alta.

Adversarial

prompt injection, jailbreak, extracción de datos. Complementa Model Armor en producción.

Los evals de CI verifican los test cases conocidos. Simulation expone los desconocidos — antes de que lo haga un usuario real.

¿Dónde está tu organización?

Nivel 1 — AI Assisted (señales)

- Algunos equipos usan Claude Code, Gemini o ChatGPT individualmente
- No hay estandarización de herramientas ni prompts entre equipos
- Los agentes son POCs que no superan el estado de notebook
- Sin gobernanza: sin registry, sin identidad, sin Model Armor
- Nadie sabe cuántos agentes existen ni qué datos acceden

Hacia Nivel 2 — AI Native (90 días)

Semanas 1-2

ADK como framework estándar · primer deploy en Agent Runtime

Semanas 3-4

Agent Identity + Agent Registry para los agentes existentes

Semanas 5-8

Model Armor activado · Agent Gateway en VPC privada

Semanas 9-12

Framework de evals · Observability · Agent Optimizer

NowVertical acompaña cada fase — desde la gobernanza mínima hasta la operación de flota multi-agente enterprise.

¿Cuál es el cuello de botella en tu organización hoy?

BUILD

¿Tenemos un framework
standardizado de agentes?

SCALE

¿Podemos escalar agentes
sin gestión de infra?

GOVERN

¿Tenemos registry,
identidad
y control de acceso?

OPTIMIZE

¿Medimos la calidad
de nuestros agentes?

— Q & A —